Assignment 4 — GitHub Actions CI Pipeline Bug Report & Fixed Workflow for ML Model CI

Student: Yosef Selim | Course: MLOps | Date: March 2026

This report identifies all bugs found in the provided broken GitHub Actions YAML pipeline, explains why each one causes a failure, and shows the corrected code. The fixed pipeline runs on every push to any branch except main, performs a linter check, validates the model environment, and uploads README.md as a GitHub artifact.

1. The Original Broken YAML

The YAML file provided in the assignment contains **5 bugs** that would cause the pipeline to either fail immediately or not behave as required. The original code is shown below exactly as given:

```
name: ML Model CI
on:
push:
branches: main
pull_request:
jobs:
validate-and-test:
runs-on: ubuntu-latest
steps:
- name: Set up Python
  uses: actions/setup-python@v5
  with:
    python-version: '3.10'
- name: Install Dependencies
  run: pip install -r requirements.txt
- name: Linter Check
- name: Model Dry Test
  run: |
    python -c "import torch; print('Model environment ready!')"
```

2. Bug Summary

#	Bug	Location	Severity	Status
1	Missing checkout step	steps (first)	CRITICAL	**FIXED**
2	No indentation / flat YAML	entire file	CRITICAL	**FIXED**
3	Linter step has no run command	Linter Check step	HIGH	**FIXED**
4	Wrong trigger — runs only on main	on: push: branches:	MEDIUM	**FIXED**
5	No artifact upload step	end of steps	MEDIUM	**ADDED**

Table 1 — All identified bugs, their location, severity, and resolution status.

3. Detailed Bug Analysis & Fixes

Bug 1 — Missing Checkout Step (CRITICAL)

The pipeline jumps straight to setting up Python without first checking out the repository code. This means **train.py, requirements.txt, and README.md are never downloaded onto the runner**. Every subsequent step that references these files will fail with "file not found".

```
# MISSING entirely — no checkout step exists
```

```
- name: Checkout Repository
  uses: actions/checkout@v4
```

Bug 2 — No Indentation (CRITICAL)

YAML is **whitespace-sensitive**. The original file has no indentation at all — every line is at column 0. GitHub Actions requires a strict nested structure: *on* → *push* → *branches* and *jobs* → *job-name* → *steps* → *step fields*. Without correct indentation the YAML parser cannot build the tree and rejects the file entirely with a syntax error before any step runs.

```
on:
push:
branches: main
jobs:
validate-and-test:
runs-on: ubuntu-latest
```

```
on:
  push:
    branches-ignore:
      - main
  pull_request:

jobs:
  validate-and-test:
    runs-on: ubuntu-latest
```

Bug 3 — Linter Check Step Has No run Command (HIGH)

A GitHub Actions step **must have either a *uses* or a *run* field**. The Linter Check step has only a name and nothing else. The runner does not know what to execute and will throw a validation error: *"Every step must define a 'uses' or 'run' key"*. The fix installs flake8 and runs it against train.py.

```
- name: Linter Check
  # no run or uses key -- invalid step
```

```
- name: Linter Check
  run: pip install flake8 && flake8 train.py --max-line-length=120 --ignore=E501
```

Bug 4 — Trigger Runs Only on main (MEDIUM)

The assignment requires the pipeline to run on **every push to all branches EXCEPT main**. The original trigger *branches: main* does the opposite — it only fires on pushes to main and ignores every feature branch. The fix uses *branches-ignore* which inverts the filter.

```
on:
  push:
    branches: main    # only fires on main -- wrong
```

```
on:
  push:
    branches-ignore:
      - main          # fires on ALL branches except main
  pull_request:       # also fires on any pull request
```

Bug 5 — No Artifact Upload Step (MEDIUM)

The assignment requires a final step that uploads README.md as a GitHub artifact named **project-doc**. This step was completely absent from the original file. The fix adds it using the official *actions/upload-artifact@v4* action.

```
# No artifact upload step exists in the original file
```

```
- name: Upload project-doc Artifact
  uses: actions/upload-artifact@v4
  with:
    name: project-doc
    path: README.md
```

4. Complete Fixed YAML File

Below is the complete corrected **ml-pipeline.yml** with all 5 bugs fixed:

```yaml
# .github/workflows/ml-pipeline.yml
name: ML Model CI

on:
  push:
    branches-ignore:
      - main
  pull_request:

jobs:
  validate-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository        # FIX 1: added missing checkout
        uses: actions/checkout@v4

      - name: Set up Python              # FIX 2: correct indentation
        uses: actions/setup-python@v5
        with:
          python-version: '3.10'

      - name: Install Dependencies
        run: pip install -r requirements.txt

      - name: Linter Check               # FIX 3: added run command
        run: pip install flake8 && flake8 train.py --max-line-length=120

      - name: Model Dry Test
        run: |
          python -c "import torch; print('Model environment ready!')"

      - name: Upload project-doc Artifact  # FIX 5: added artifact upload
        uses: actions/upload-artifact@v4
        with:
          name: project-doc
          path: README.md
```

5. Step-by-Step Setup Instructions

Step 1 — Create a GitHub repository

Go to github.com → New Repository → name it e.g. *mlops-assignment3* → set it Public → Create.

Step 2 — Upload your project files

Push these files to the repository root:

```
train.py
requirements.txt
README.md
```

Step 3 — Create the workflow file

In your repository, create the folder path **.github/workflows/** and inside it create the file **ml-pipeline.yml** with the fixed content from Section 4.

Step 4 — Trigger the pipeline

Create a new branch (not main) and push to it. Because the trigger is *branches-ignore: main*, any push to a feature branch fires the pipeline.

```
git checkout -b feature/test-pipeline
git add .
git commit -m 'test pipeline'
git push origin feature/test-pipeline
```

Step 5 — Check the Actions tab

Go to your repository on GitHub → click the **Actions** tab → you will see the workflow running. Wait for all steps to turn green. Take a screenshot of this for your submission.

Step 6 — Download the artifact

After the run succeeds, click into the run → scroll to the bottom → under **Artifacts** you will see *project-doc* available for download. This confirms the upload step worked correctly.

Screenshot placeholder: After your pipeline runs successfully, paste a screenshot of the GitHub Actions tab showing a green checkmark here before final submission.

[Paste your GitHub Actions green run screenshot here]